CONTOSO RETAIL DW

End-to-End BI Analytics Project

Document 3 of 3 — Technical Reference Appendices

Appendix A: Script Execution Order | Appendix B: Column Dictionary | Appendix C: Relationship Map | Appendix D: Full DAX Formula Library (49 Measures)

Field	Detail
Author	Waleed Mouhammed
Programme	DEPI — Data Analysis Track
Document	3 of 3 — Technical Reference Appendices
Version	1.0 — March 2026
Contents	Script dependency chain, complete column dictionary, all 36 model relationships, full DAX formula code for all 49 measures

Appendix A: Script Execution Order & Dependency Chain

The table below is the authoritative reference for script execution. Scripts must be run in strict sequential order. Each script from Script 01 onwards contains SET NOEXEC ON pre-execution guards that halt execution and print an informative error message if any dependency is missing. All scripts are fully idempotent — they can be re-run on a live database without error or data corruption.

Order	Script File	Purpose	Dependencies	Key Output	Re-Runnable?
1	00_create_schemas_v2.sql	Creates the three schema namespaces that all subsequent scripts depend on	None — only requires ContosoRetailDW database to exist	[gen], [dim], [fact] schemas (owned by dbo)	✓ Yes — IF NOT EXISTS guards prevent duplicate creation
2	01_gen_ReferenceDimensions_v2.sql	Creates and populates three small reference dimension tables that all gen scripts reference for FK lookups	Script 00 — [gen] schema must exist	gen.DimAcquisitionChannel (7 rows), gen.DimPaymentMethod (6 rows), gen.DimReturnReason (8 rows)	✓ Yes — DROP + CREATE pattern with IF OBJECT_ID guards
3	02_gen_CustomerAcquisition_v2.sql	Assigns each qualifying customer to exactly one acquisition channel using a 5-factor multiplicative scoring model	Script 01 (gen.DimAcquisitionChannel must exist); dbo.DimCustomer and dbo.FactOnlineSales must exist	gen.CustomerAcquisition — one row per qualifying Person-type customer	✓ Yes — TRUNCATE + INSERT pattern inside idempotent guard
4	03_gen_OrderPayment_v2.sql	Assigns a payment method to each online order using era-accurate probability weighting	Script 01 (gen.DimPaymentMethod must exist); dbo.FactOnlineSales must exist	gen.OrderPayment — one row per distinct SalesOrderNumber in FactOnlineSales	✓ Yes — TRUNCATE + INSERT pattern
5	04_gen_OrderFulfillment_v2.sql	Generates a complete fulfilment lifecycle for every online order — ship date, delivery date, SLA status, shipping method	dbo.FactOnlineSales, dbo.DimCustomer, dbo.DimStore must all exist; Script 01 not required (no FK to gen reference tables)	gen.OrderFulfillment — one row per distinct SalesOrderNumber in FactOnlineSales	✓ Yes — TRUNCATE + INSERT pattern
6	05_gen_FactMarketingSpend_v2.sql	Generates monthly marketing spend by channel, internally calibrated against actual new-customer counts from Script 02	Script 02 (gen.CustomerAcquisition must be populated with actual counts); Script 01 (gen.DimAcquisitionChannel)	gen.FactMarketingSpend — one row per (calendar month × acquisition channel), approximately 252 rows total	✓ Yes — TRUNCATE + INSERT pattern; must be re-run if Script 02 output changes
7	06_gen_FactCustomerSurvey_v2.sql	Generates behaviourally coherent NPS and CSAT survey responses correlated with actual transaction history	dbo.DimCustomer, dbo.FactOnlineSales; Script 01 not required	gen.FactCustomerSurvey — one row per survey response (response rate ~15–30% per customer)	✓ Yes — TRUNCATE + INSERT pattern
8	07_gen_OnlineReturnEvents_v2.sql	Generates event-level return records for the online channel with return dates, quantities, amounts, reasons, and lag days	dbo.FactOnlineSales, dbo.DimCustomer; Script 01 (gen.DimReturnReason must exist for FK)	gen.OnlineReturnEvents — one row per online return event	✓ Yes — TRUNCATE + INSERT pattern

Order	Script File	Purpose	Dependencies	Key Output	Re-Runnable?
9	08_gen_PhysicalReturnEvents_v2.sql	Generates event-level return records for the physical store channel — CustomerKey and SalesOrderNumber are absent by design (FactSales has no customer grain)	dbo.FactSales; Script 01 (gen.DimReturnReason must exist)	gen.PhysicalReturnEvents — one row per physical store return event	✓ Yes — TRUNCATE + INSERT pattern
10	09_dim_Views_v1.sql	Creates all 11 analytical dimension views in the [dim] schema — the semantic dimension layer for Power BI	Scripts 00–08 must all be complete; all required dbo tables (DimDate, DimCustomer, DimProduct, DimProductSubcategory, DimProductCategory, DimStore, DimEmployee, DimPromotion, DimCurrency, DimChannel, DimGeography) must exist	11 [dim] views: dim.vDate, dim.vCustomer, dim.vProduct, dim.vStore, dim.vPromotion, dim.vEmployee, dim.vPaymentMethod, dim.vAcquisitionChannel, dim.vCurrency, dim.vReturnReason, dim.vChannel	✓ Yes — CREATE OR ALTER VIEW; fully safe to re-run on live database
11	10_fact_Views_v2.sql	Creates all 11 analytical fact views in the [fact] schema — the semantic measurement layer for Power BI	Script 09 must be complete (all dim views must exist before fact views can reference them); Scripts 00–08 must all be complete	11 [fact] views: fact.vOnlineSales, fact.vStoreSales, fact.vReturns, fact.vInventory, fact.vSalesQuota, fact.vExchangeRate, fact.vOrderFulfillment, fact.vCustomerSurvey, fact.vMarketingSpend, fact.vCustomerAcquisition, fact.vOrderPayment	✓ Yes — CREATE OR ALTER VIEW; fully safe to re-run
12	export_contoso_views_v7.py	Reads all 22 views from SQL Server and exports them as Snappy-compressed Parquet files; generates SchemaDictionary.csv	Scripts 00–10 all complete; Python 3.9+ with pyodbc, sqlalchemy, pandas, pyarrow installed; ODBC Driver 18 for SQL Server configured; output SharePoint directory accessible	22 Parquet files + SchemaDictionary.csv in /Contoso Data Files/ directory	✓ Yes — overwrites existing Parquet files; safe to re-run after any view change

Critical Execution Note

If any [gen] data generation script (Scripts 02–08) is modified and re-run, Script 05 (gen.FactMarketingSpend) must also be re-run because its spend figures are calibrated against the actual new-customer counts produced by Script 02. After any SQL change, the Python export pipeline must be re-run to refresh the Parquet files consumed by Power BI.

Appendix B: Key Column Dictionary

This appendix documents the most analytically significant columns across the 22 views. Columns marked FK are foreign keys used to establish Power BI model relationships. Columns marked Hidden are set isHidden=true in the TMDL and are invisible to report authors in the Fields pane. All DateKey columns are INT YYYYMMDD format per the universal date key contract.

B.1 — dim.vDate

Column	SQL Type	Role	Description & Notes
DateKey	INT	PK — conformed date key for all relationships	YYYYMMDD format. Shifted +16 years from source. e.g., 20250115 for January 15, 2025. Primary FK target for all date relationships.
Date	DATE	Mark as Date Table column	Used for Power BI 'Mark as Date Table'. Required for DATESYTD, SAMEPERIODLASTYEAR, and all built-in Time Intelligence functions to work correctly.
CalendarYear	INT	Slicer / filter	Shifted year: 2021–2027. Core analytical range: 2023–2025.
CalendarMonthNumber	INT	Sort / hierarchy	1–12. Used as sort column for CalendarMonthName.
CalendarMonthName	VARCHAR	Label / slicer	January through December. Requires CalendarMonthNumber as sort column.
CalendarQuarterNumber	INT	Hierarchy	1–4.
FiscalYear	INT	Fiscal filter	July-start fiscal year. FY2024 = July 2023 – June 2024.
FiscalQuarterNumber	INT	Fiscal hierarchy	FQ1 = July–September; FQ2 = October–December; FQ3 = January–March; FQ4 = April–June.
FiscalMonthNumber	INT	Fiscal hierarchy	FM1 = July through FM12 = June.
IsWeekend	BIT	Operational filter	1 = Saturday or Sunday.
IsWorkDay	BIT	Operational filter	From dbo.DimDate source. 1 = business day.
IsHoliday	BIT	Seasonal filter	From dbo.DimDate source.
IsCurrentYear	BIT	Dynamic display flag	GETDATE()-based. 1 if YEAR(GETDATE()) = CalendarYear. Changes daily — not used in historical measures.
IsCurrentMonth	BIT	Dynamic display flag	GETDATE()-based. Changes monthly.
IsToday	BIT	Dynamic display flag	GETDATE()-based. Exactly 1 row is 1 at any given time.
NorthAmericaSeason	VARCHAR	Seasonality slicer	Spring / Summer / Autumn / Winter — North American calendar. Source: dbo.DimDate.
EuropeSeason	VARCHAR	Seasonality slicer	Spring / Summer / Autumn / Winter — European calendar.
AsiaSeason	VARCHAR	Seasonality slicer	Spring / Summer / Autumn / Winter — Asian calendar.

B.2 — dim.vCustomer

Column	SQL Type	Role / Hidden	Description & Notes
CustomerKey	INT	PK — FK target for all customer relationships	Includes Person AND Company types. CustomerType filter removed after 9.7M orphan key discovery.
FullName	NVARCHAR	Visible	CONCAT_WS(FirstName, MiddleName, LastName) — NULL-safe. Company rows show CompanyName via separate logic.
CustomerType	NVARCHAR	Slicer	'Person' or 'Company'. Primary B2B/B2C split filter.
CompanyName	NVARCHAR	Visible	Populated for Company-type customers only. NULL for Person customers.
Gender	NVARCHAR	Slicer / demographic	'Male' / 'Female' / 'Unknown' / 'N/A' (B2B). Normalised from source M/F codes.
MaritalStatus	NVARCHAR	Slicer / demographic	'Married' / 'Single' / 'Divorced' / 'Widowed' / 'Unknown' / 'N/A'. Normalised from M/S/D/W codes.
BirthDate	DATE	NOT shifted — fixed biological attribute	NOT shifted +16 years. Age computed as DATEDIFF(YEAR, BirthDate, '2025-12-31').
Age	INT	Computed demographic	Age as of '2025-12-31'. NULL for Company customers.
AgeGroup	NVARCHAR	Slicer / demographic	'18–25' / '26–35' / '36–45' / '46–55' / '56–65' / '65+'. NULL for Company.
YearlyIncome	MONEY	Numeric attribute	Annual income in USD. NULL for Company customers.
IncomeGroup	NVARCHAR	Slicer / segment	'Low' (<$30K) / 'Lower-Middle' (<$60K) / 'Upper-Middle' (<$100K) / 'High' (≥$100K). NULL for Company.
LifeStage	NVARCHAR	CMO segment slicer	'Young Single' / 'Young Couple' / 'Young Family' / 'Mature Single/Couple' / 'Mature Family' / 'Senior'. Based on age + marital status + children.
DateFirstPurchase	DATE	Acquisition analytics	NOT shifted. Used for CustomerTenureYears calculation. Source of AcquisitionDate in gen.CustomerAcquisition.
CustomerTenureYears	DECIMAL(5,2)	Computed tenure	DATEDIFF from DateFirstPurchase to '2025-12-31' in fractional years. NULL for Company.
CustomerLifecycleStage	NVARCHAR	Lifecycle slicer	'New' (≤1yr) / 'Developing' (≤2yr) / 'Established' (≤3yr) / 'Loyal' (>3yr).
CustomerContinent	NVARCHAR	Geography hierarchy — slicer	Prefixed 'Customer' to avoid collision with StoreContinent in the same model.
CustomerCountry	NVARCHAR	Geography — map visual	Prefixed 'Customer'. Used for choropleth map and geographic revenue analysis.
CustomerState	NVARCHAR	Geography hierarchy	Prefixed 'Customer'.
CustomerCity	NVARCHAR	Geography drill-down	Prefixed 'Customer'. Lowest geography level available.

B.3 — dim.vProduct

Column	SQL Type	Role	Description & Notes
ProductKey	INT	PK — FK target for all product relationships	Surrogate key from dbo.DimProduct.
ProductName	NVARCHAR	Label / slicer	Full product name for display in visuals.
ProductLabel	NVARCHAR	SKU identifier	Product code / SKU from source system.
CategoryName	NVARCHAR	Hierarchy Level 1 — slicer	Top-level product grouping. 8 categories total.
SubcategoryName	NVARCHAR	Hierarchy Level 2 — slicer	Mid-level product grouping. 44 subcategories total.
ProductBreadcrumb	NVARCHAR	Drill-through label	Category > Subcategory > Product — useful for tooltip and drill-through context.
BrandName	NVARCHAR	Brand slicer	Brand attribution for brand performance analysis.
Manufacturer	NVARCHAR	Supplier analytics	Manufacturer name.
ColorName	NVARCHAR	Product attribute slicer	Colour for electronics accessories and devices.
CatalogCost	MONEY	List cost price	Renamed from source UnitCost — signals list-price semantics, distinct from transactional TotalCost in fact tables.
CatalogPrice	MONEY	List sell price	Renamed from source UnitPrice — signals list-price semantics, distinct from transactional UnitPrice in fact views.
CatalogGrossMarginPct	DECIMAL(10,4)	Pre-computed margin	(CatalogPrice - CatalogCost) / NULLIF(CatalogPrice, 0). List-price margin at SKU level.
AvailableForSaleDate	DATE	Lifecycle date — shifted	Product launch date shifted +16 years from source.
StopSaleDate	DATE	Lifecycle date — shifted	Discontinuation date shifted +16 years. NULL = still active.
ProductStatus	NVARCHAR	Status slicer	'Active' (StopSaleDate IS NULL or > '2025-12-31') / 'Discontinued'. Computed from shifted dates.
CatalogueAgeYears	INT	Product age metric	DATEDIFF(YEAR, AvailableForSaleDate, '2025-12-31'). Used in product lifecycle analysis.

B.4 — fact.vOnlineSales (Primary Analytical Fact)

Column	SQL Type	Role / Hidden	Description & Notes
OnlineSalesKey	INT	PK — Hidden	Surrogate key from dbo.FactOnlineSales. Hidden — never used in visuals.
DateKey	INT	FK → dim.vDate[DateKey] — Hidden — ACTIVE relationship	YYYYMMDD INT. Shifted +16 years. The join column for all date-based filtering.
OrderDate	DATE	Companion display column — Visible	Shifted display date. NOT used as FK — DateKey INT is used for all relationships.

Column	SQL Type	Role / Hidden	Description & Notes
CustomerKey	INT	FK → dim.vCustomer[CustomerKey] — Hidden — ACTIVE	The ONLY FK to dim.vCustomer in the entire model. Gateway for all CLV, cohort, and segmentation analytics.
ProductKey	INT	FK → dim.vProduct[ProductKey] — Hidden — ACTIVE	
StoreKey	INT	FK → dim.vStore[StoreKey] — Hidden — ACTIVE	Online sales are associated with 'online store' entities in DimStore.
PromotionKey	INT	FK → dim.vPromotion[PromotionKey] — Hidden — ACTIVE	Key=1 = No Promotion. Must be handled in DAX for 'promotion-only' filters.
CurrencyKey	INT	FK → dim.vCurrency[CurrencyKey] — Hidden — ACTIVE	
SalesOrderNumber	NVARCHAR(20)	Degenerate dimension — Visible	Natural key from source system. Used for basket analysis (GROUP BY SalesOrderNumber) and TREATAS bridges to Fulfillment and Payment facts. Not a FK in the model.
SalesOrderLineNumber	INT	Degenerate dimension — Visible	Line-item identifier within an order. Used with SalesOrderNumber for unique order-line identification.
SalesQuantity	INT	Additive measure	Units sold at line-item level.
SalesAmount	MONEY	Additive measure	Gross revenue = SalesQuantity × UnitPrice at time of sale.
ReturnQuantity	INT	Additive measure	Embedded returns in the source line. For event-level returns use fact.vReturns.
ReturnAmount	MONEY	Additive measure	Embedded return value in the source line.
DiscountAmount	MONEY	Additive measure	Discount applied at line level.
DiscountQuantity	INT	Additive measure	Units discounted.
TotalCost	MONEY	Additive measure	Cost of goods at time of sale = SalesQuantity × UnitCost.
UnitCost	MONEY	Transactional price attribute	Cost per unit AT TIME OF SALE. Distinct from dim.vProduct[CatalogCost] which is the current list price.
UnitPrice	MONEY	Transactional price attribute	Price per unit AT TIME OF SALE. Distinct from dim.vProduct[CatalogPrice].
GrossProfit	MONEY	Pre-computed additive measure	SalesAmount − TotalCost. Pre-computed in SQL to avoid repeated DAX subtraction. Additive across any aggregation.

B.5 — fact.vReturns (Unified Return Fact)

Column	SQL Type	Role	Description & Notes
ReturnEventID	VARCHAR	PK (derived)	Prefixed surrogate: 'ONL-nnn' for online rows, 'PHY-nnn' for physical rows. Guarantees namespace uniqueness across the UNION ALL without an IDENTITY column.
ReturnDateKey	INT	FK → dim.vDate[DateKey] — ACTIVE relationship	Primary date dimension for returns — 'returns processed in this period'.

Column	SQL Type	Role	Description & Notes
ReturnDate	DATE	Companion display column	Shifted return date for visual display.
OriginalSaleDateKey	INT	FK → dim.vDate[DateKey] — INACTIVE relationship	The sale date of the item being returned. Activated in DAX via USERELATIONSHIP() for return-lag cohort analysis: 'returns from items sold in this period'.
OriginalSaleDate	DATE	Companion display column	Shifted original sale date for display.
CustomerKey	INT	FK → dim.vCustomer — NULL for Physical rows	Online rows: populated (NOT NULL). Physical rows: NULL BY DESIGN — FactSales has no customer grain. This is architectural, not a data quality issue.
StoreKey	INT	FK → dim.vStore — NULL for Online rows	Physical rows: populated. Online rows: NULL BY DESIGN — online orders have no physical store.
ProductKey	INT	FK → dim.vProduct — Always populated	Product being returned. Populated for both channels.
ReturnReasonKey	INT	FK → dim.vReturnReason	Links to IsOperationalFailure flag. Populated for both channels.
ReturnChannel	VARCHAR	Discriminator	'Online' or 'Physical'. Primary column for channel-split return analysis.
SalesOrderNumber	NVARCHAR	Cross-fact bridge — Online rows only	Populated for Online rows — TREATAS bridge key to fact.vOnlineSales. NULL for Physical rows (FactSales has no order grain).
ReturnQuantity	INT	Additive measure	Units returned in this return event.
ReturnAmount	MONEY	Additive measure	Value of returned merchandise.
RefundAmount	MONEY	Additive measure	Cash actually refunded. May differ from ReturnAmount if a restocking fee was applied.
IsRefunded	BIT	Boolean measure / filter	1 = refund issued; 0 = exchange, store credit, or no refund.
ReturnLagDays	INT	Operational metric	Days between OriginalSaleDate and ReturnDate. Measures return speed — fast lag = strong dissatisfaction; slow lag = policy-driven or deliberate return.

B.6 — fact.vOrderFulfillment

Column	SQL Type	Role	Description & Notes
FulfillmentKey	INT	PK	Surrogate key for the fulfillment record.
SalesOrderNumber	NVARCHAR	TREATAS bridge key	Natural key matching fact.vOnlineSales[SalesOrderNumber]. Used in TREATAS patterns to link fulfilment metrics back to customer/product/promotion context.
OrderDateKey	INT	FK → dim.vDate — ACTIVE	Primary date for 'orders placed in this period' filtering.
ShipDateKey	INT	FK → dim.vDate — INACTIVE	Activated via USERELATIONSHIP() for 'orders shipped in this period' analysis.
DeliveryDateKey	INT	FK → dim.vDate — INACTIVE	Activated via USERELATIONSHIP() for 'orders delivered in this period' analysis.
ShippingMethod	NVARCHAR	Slicer / category	'Standard' / 'Express' / 'Overnight'. Volume-weighted assignment in generation.

Column	SQL Type	Role	Description & Notes
FulfillmentStatus	NVARCHAR	Status slicer	'Delivered' / 'In Transit' / 'Processing' etc.
GeoDistanceCategory	NVARCHAR	Geographic segment	'Domestic' / 'Continental' / 'International'. Drives transit time SLA thresholds.
ProcessingDays	INT	Additive metric	Warehouse handling time: OrderDate to ShipDate.
TransitDays	INT	Additive metric	Carrier time: ShipDate to DeliveryDate.
TotalFulfillmentDays	INT	Additive metric	End-to-end customer-facing lead time: OrderDate to DeliveryDate.
IsOnTime	BIT	Primary SLA flag	1 = delivered within SLA window for this ShippingMethod and GeoDistanceCategory.
SLATier	NVARCHAR	Delay severity	'On Time' / 'Minor Delay' / 'Significant Delay' / 'Critical Delay'. Enables SLA severity distribution analysis.

Appendix C: Complete Power BI Relationship Map

All 36 Power BI relationships are documented below. Model rules: all relationships are Many-to-One (fact → dimension), all filter directions are Single (dimension filters fact), and bi-directional filtering is disabled on every relationship. Active relationships are used by default for all visual filtering. Inactive relationships must be activated explicitly in DAX measures via USERELATIONSHIP().

Why No Bi-Directional Relationships?
Bi-directional relationships allow a filter on one fact table to propagate through a shared dimension into another fact table, producing incorrect aggregations across unrelated facts. For example, filtering fact.vOnlineSales by ProductKey would incorrectly filter fact.vInventory through the shared dim.vProduct relationship if bi-directional were enabled. Single-direction filtering preserves the analytical independence of each fact table — a fundamental Kimball correctness requirement.

#	From Table	From Column	To Table	To Column	Cardinality	Filter Direction	Active?	Purpose
1	fact.vOnlineSales	DateKey	dim.vDate	DateKey	Many:1	Single →	✓ Active	Primary date for online sales filtering
2	fact.vOnlineSales	CustomerKey	dim.vCustomer	CustomerKey	Many:1	Single →	✓ Active	Customer context for CLV and segmentation
3	fact.vOnlineSales	ProductKey	dim.vProduct	ProductKey	Many:1	Single →	✓ Active	Product context for category analysis
4	fact.vOnlineSales	StoreKey	dim.vStore	StoreKey	Many:1	Single →	✓ Active	Store/geography context for online orders
5	fact.vOnlineSales	PromotionKey	dim.vPromotion	PromotionKey	Many:1	Single →	✓ Active	Promotion effectiveness analysis
6	fact.vOnlineSales	CurrencyKey	dim.vCurrency	CurrencyKey	Many:1	Single →	✓ Active	Multi-currency revenue normalisation
7	fact.vStoreSales	DateKey	dim.vDate	DateKey	Many:1	Single →	✓ Active	Primary date for store sales filtering
8	fact.vStoreSales	ProductKey	dim.vProduct	ProductKey	Many:1	Single →	✓ Active	Product context for physical store sales
9	fact.vStoreSales	StoreKey	dim.vStore	StoreKey	Many:1	Single →	✓ Active	Store and geography context
10	fact.vStoreSales	PromotionKey	dim.vPromotion	PromotionKey	Many:1	Single →	✓ Active	In-store promotion analysis
11	fact.vStoreSales	CurrencyKey	dim.vCurrency	CurrencyKey	Many:1	Single →	✓ Active	Multi-currency store revenue
12	fact.vStoreSales	ChannelKey	dim.vChannel	ChannelKey	Many:1	Single →	✓ Active	Distribution channel attribution — unique to this fact
13	fact.vReturns	ReturnDateKey	dim.vDate	DateKey	Many:1	Single →	✓ Active	'Returns processed in period X' — primary return date
14	fact.vReturns	OriginalSaleDateKey	dim.vDate	DateKey	Many:1	Single →	✗ INACTIVE	'Returns from items sold in period X' — activated via USERELATIONSHIP() for cohort analysis

#	From Table	From Column	To Table	To Column	Cardinality	Filter Direction	Active?	Purpose
15	fact.vReturns	CustomerKey	dim.vCustomer	CustomerKey	Many:1	Single →	✓ Active	Online return customer context (NULL for physical rows by design)
16	fact.vReturns	ProductKey	dim.vProduct	ProductKey	Many:1	Single →	✓ Active	Returned product context — enables return rate by category
17	fact.vReturns	StoreKey	dim.vStore	StoreKey	Many:1	Single →	✓ Active	Physical return store context (NULL for online rows by design)
18	fact.vReturns	ReturnReasonKey	dim.vReturnReason	ReturnReasonKey	Many:1	Single →	✓ Active	Return reason breakdown — IsOperationalFailure analysis
19	fact.vInventory	DateKey	dim.vDate	DateKey	Many:1	Single →	✓ Active	Inventory snapshot date — daily grain
20	fact.vInventory	ProductKey	dim.vProduct	ProductKey	Many:1	Single →	✓ Active	Product context for inventory metrics
21	fact.vInventory	StoreKey	dim.vStore	StoreKey	Many:1	Single →	✓ Active	Store-level inventory context
22	fact.vSalesQuota	DateKey	dim.vDate	DateKey	Many:1	Single →	✓ Active	Quota period date
23	fact.vSalesQuota	EmployeeKey	dim.vEmployee	EmployeeKey	Many:1	Single →	✓ Active	Employee-level quota assignment
24	fact.vExchangeRate	DateKey	dim.vDate	DateKey	Many:1	Single →	✓ Active	Monthly exchange rate date
25	fact.vExchangeRate	CurrencyKey	dim.vCurrency	CurrencyKey	Many:1	Single →	✓ Active	Currency-to-exchange rate lookup
26	fact.vOrderFulfillment	OrderDateKey	dim.vDate	DateKey	Many:1	Single →	✓ Active	'Orders placed in period X' — primary fulfillment date
27	fact.vOrderFulfillment	ShipDateKey	dim.vDate	DateKey	Many:1	Single →	✗ INACTIVE	'Orders shipped in period X' — activated via USERELATIONSHIP()
28	fact.vOrderFulfillment	DeliveryDateKey	dim.vDate	DateKey	Many:1	Single →	✗ INACTIVE	'Orders delivered in period X' — activated via USERELATIONSHIP()
29	fact.vCustomerSurvey	SurveyDateKey	dim.vDate	DateKey	Many:1	Single →	✓ Active	Survey response date for NPS trend analysis
30	fact.vCustomerSurvey	CustomerKey	dim.vCustomer	CustomerKey	Many:1	Single →	✓ Active	Customer context for satisfaction segmentation
31	fact.vMarketingSpend	MonthStartDateKey	dim.vDate	DateKey	Many:1	Single →	✓ Active	Monthly marketing spend date
32	fact.vMarketingSpend	AcquisitionChannelKey	dim.vAcquisitionChannel	AcquisitionChannelKey	Many:1	Single →	✓ Active	Channel-level spend and CAC analysis
33	fact.vCustomerAcquisition	CustomerKey	dim.vCustomer	CustomerKey	Many:1	Single →	✓ Active	Bridge: customer to acquisition channel via this fact
34	fact.vCustomerAcquisition	AcquisitionChannelKey	dim.vAcquisitionChannel	AcquisitionChannelKey	Many:1	Single →	✓ Active	Bridge: acquisition channel to customer via this fact

#	From Table	From Column	To Table	To Column	Cardinality	Filter Direction	Active?	Purpose
35	fact.vCustomerAcquisition	AcquisitionDateKey	dim.vDate	DateKey	Many:1	Single →	✓ Active	Acquisition date for 'customers acquired in period X' analysis
36	fact.vOrderPayment	PaymentMethodKey	dim.vPaymentMethod	PaymentMethodKey	Many:1	Single →	✓ Active	Bridge: online order to payment method

C.2 — Cross-Fact TREATAS Bridges (Not Model Relationships)

Three fact-to-fact analytical connections exist as TREATAS patterns in DAX rather than formal model relationships. These connections link secondary facts back to the primary fact (fact.vOnlineSales) via the shared SalesOrderNumber degenerate dimension.

From Fact	To Fact	Bridge Column	DAX TREATAS Pattern	Analytical Use
fact.vOnlineSales	fact.vOrderFulfillment	SalesOrderNumber	CALCULATE([Avg Fulfillment Days], TREATAS(VALUES(fact_vOnlineSales[SalesOrderNumber]), fact_vOrderFulfillment[SalesOrderNumber]))	Filter fulfilment metrics to only orders matching a customer segment, product, or promotion context
fact.vOnlineSales	fact.vOrderPayment	SalesOrderNumber	CALCULATE([Payment Method Count], TREATAS(VALUES(fact_vOnlineSales[SalesOrderNumber]), fact_vOrderPayment[SalesOrderNumber]))	Filter payment method distribution to orders matching a specific customer or product context
fact.vCustomerSurvey	fact.vCustomerAcquisition	CustomerKey	CALCULATE([NPS Score], TREATAS(CALCULATETABLE(VALUES(fact_vCustomerAcquisition[CustomerKey]), channel filter), fact_vCustomerSurvey[CustomerKey]))	Filter NPS score to customers acquired via a specific channel — [NPS by Acquisition Channel] measure

Appendix D: Full DAX Formula Library (49 Measures)

This appendix contains the complete, ready-to-implement DAX formula for all 49 measures across the seven analytical pipelines. All measures follow the VAR/RETURN pattern, use DIVIDE() for all division operations, and reference base measures via named variables to avoid repeated sub-expression evaluation. Table and column references use underscore notation (fact_vOnlineSales) as Power BI replaces spaces and dots with underscores in the TMDL.

> **Implementation Note**
> Measures should be implemented in pipeline order (P1 first, P7 last) because higher pipelines reference measures from lower pipelines. For example, [Net Revenue] in P1 is referenced by [Gross Margin %] in P1, [CAC-to-CLV Ratio] in P3, and [Marketing Spend % of Revenue] in P3. Building P1 first ensures all dependencies exist before dependent measures are created.

Pipeline P1 — Core Revenue

[Total Online Revenue]

```
[Total Online Revenue] =
    SUM(fact_vOnlineSales[SalesAmount])
```

[Total Store Revenue]

```
[Total Store Revenue] =
    SUM(fact_vStoreSales[SalesAmount])
```

[Total Revenue]

```
[Total Revenue] =
    VAR OnlineRev = [Total Online Revenue]
    VAR StoreRev  = [Total Store Revenue]
    RETURN
        OnlineRev + StoreRev
```

[Total Discounts]

```
[Total Discounts] =
    VAR OnlineDisc = SUM(fact_vOnlineSales[DiscountAmount])
    VAR StoreDisc  = SUM(fact_vStoreSales[DiscountAmount])
    RETURN
        OnlineDisc + StoreDisc
```

[Total Returns Amount (Embedded)]

```
[Total Returns Amount (Embedded)] =
    VAR OnlineRet = SUM(fact_vOnlineSales[ReturnAmount])
    VAR StoreRet  = SUM(fact_vStoreSales[ReturnAmount])
    RETURN
        OnlineRet + StoreRet
```

[Net Revenue]

```
[Net Revenue] =
    VAR Gross     = [Total Revenue]
    VAR Returns   = [Total Returns Amount (Embedded)]
    VAR Discounts = [Total Discounts]
    RETURN
        Gross - Returns - Discounts
```

[Total Gross Profit]

```
[Total Gross Profit] =
    VAR OnlineGP = SUM(fact_vOnlineSales[GrossProfit])
    VAR StoreGP  = SUM(fact_vStoreSales[GrossProfit])
    RETURN
        OnlineGP + StoreGP
```

[Gross Margin %]

```
[Gross Margin %] =
    VAR GP  = [Total Gross Profit]
    VAR Rev = [Total Revenue]
    RETURN
        DIVIDE(GP, Rev)
```

[Total Transactions (Online)]

```
[Total Transactions (Online)] =
    DISTINCTCOUNT(fact_vOnlineSales[SalesOrderNumber])
```

[Total Transactions]

```
[Total Transactions] =
    VAR OnlineTx = DISTINCTCOUNT(fact_vOnlineSales[SalesOrderNumber])
    VAR StoreTx  = COUNTROWS(fact_vStoreSales)
    RETURN
        OnlineTx + StoreTx
```

[Average Order Value (AOV)]

```
[Average Order Value (AOV)] =
    VAR NR = [Net Revenue]
    VAR TX = [Total Transactions]
    RETURN
        DIVIDE(NR, TX)
```

[YoY Revenue Growth %]

```
[YoY Revenue Growth %] =
    VAR CurrentRev = [Total Revenue]
    VAR PriorYearRev =
        CALCULATE(
            [Total Revenue],
            SAMEPERIODLASTYEAR(dim_vDate[Date])
        )
    RETURN
        DIVIDE(CurrentRev - PriorYearRev, PriorYearRev)
```

Pipeline P2 — Customer Lifetime Value

[Total Customers]

```
[Total Customers] =
    DISTINCTCOUNT(fact_vOnlineSales[CustomerKey])
```

[Historical CLV]

```
[Historical CLV] =
    VAR TotalRev  = [Total Online Revenue]
    VAR CustCount = [Total Customers]
    RETURN
        DIVIDE(TotalRev, CustCount)
```

[Avg Purchase Frequency]

```
[Avg Purchase Frequency] =
    VAR TxCount   = [Total Transactions (Online)]
    VAR CustCount = [Total Customers]
    RETURN
        DIVIDE(TxCount, CustCount)
```

[Repeat Customer Rate %]

```
[Repeat Customer Rate %] =
    VAR RepeatCustomers =
        CALCULATE(
            DISTINCTCOUNT(fact_vOnlineSales[CustomerKey]),
            FILTER(
                VALUES(fact_vOnlineSales[CustomerKey]),
                CALCULATE(
                    DISTINCTCOUNT(fact_vOnlineSales[SalesOrderNumber])
                ) > 1
            )
        )
    VAR TotalCust = [Total Customers]
    RETURN
        DIVIDE(RepeatCustomers, TotalCust)
```

[New Customers (Period)]

```
[New Customers (Period)] =
    VAR MinDate = MIN(dim_vDate[Date])
    VAR MaxDate = MAX(dim_vDate[Date])
    RETURN
        CALCULATE(
            DISTINCTCOUNT(fact_vCustomerAcquisition[CustomerKey]),
            fact_vCustomerAcquisition[AcquisitionDate] >= MinDate,
            fact_vCustomerAcquisition[AcquisitionDate] <= MaxDate
        )
```

[Churned Customers]

```
[Churned Customers] =
    VAR ChurnThresholdDays = 365
    VAR CutoffDate = DATE(2025, 12, 31)
```

```
    RETURN
        CALCULATE(
            DISTINCTCOUNT(fact_vOnlineSales[CustomerKey]),
            FILTER(
                VALUES(fact_vOnlineSales[CustomerKey]),
                VAR LastPurchase =
                    CALCULATE(MAX(fact_vOnlineSales[OrderDate]))
                RETURN
                    DATEDIFF(LastPurchase, CutoffDate, DAY) > ChurnThresholdDays
            )
        )
```

[Customer Retention Rate %]

```
[Customer Retention Rate %] =
    VAR Total   = [Total Customers]
    VAR Churned = [Churned Customers]
    RETURN
        DIVIDE(Total - Churned, Total)
```

[CLV by Acquisition Channel]

```
[CLV by Acquisition Channel] =
    -- Applied in dim_vAcquisitionChannel filter context
    VAR ChannelCusts =
        CALCULATETABLE(
            VALUES(fact_vCustomerAcquisition[CustomerKey])
        )
    VAR ChannelRevenue =
        CALCULATE(
            [Total Online Revenue],
            TREATAS(ChannelCusts, fact_vOnlineSales[CustomerKey])
        )
    VAR CustCount =
        COUNTROWS(ChannelCusts)
    RETURN
        DIVIDE(ChannelRevenue, CustCount)
```

Pipeline P3 — Marketing Performance

[Total Marketing Spend]

```
[Total Marketing Spend] =
    SUM(fact_vMarketingSpend[MonthlySpend])
```

[Marketing Spend % of Revenue]

```
[Marketing Spend % of Revenue] =
    VAR Spend = [Total Marketing Spend]
    VAR Rev   = [Total Revenue]
    RETURN
        DIVIDE(Spend, Rev)
```

[Customer Acquisition Cost (CAC)]

```
[Customer Acquisition Cost (CAC)] =
```

```
    VAR Spend    = [Total Marketing Spend]
    VAR NewCusts = SUM(fact_vMarketingSpend[NewCustomersAcquired])
    RETURN
        DIVIDE(Spend, NewCusts)
```

[Return on Ad Spend (ROAS)]

```
[Return on Ad Spend (ROAS)] =
    VAR RevDriven = SUM(fact_vMarketingSpend[EstimatedRevenueDriven])
    VAR Spend     = [Total Marketing Spend]
    RETURN
        DIVIDE(RevDriven, Spend)
```

[Cost Per Click (CPC)]

```
[Cost Per Click (CPC)] =
    VAR Spend  = [Total Marketing Spend]
    VAR Clicks = SUM(fact_vMarketingSpend[Clicks])
    RETURN
        DIVIDE(Spend, Clicks)
```

[CAC-to-CLV Ratio]

```
[CAC-to-CLV Ratio] =
    VAR CLV = [Historical CLV]
    VAR CAC = [Customer Acquisition Cost (CAC)]
    RETURN
        DIVIDE(CLV, CAC)
```

[Paid Customers Mix %]

```
[Paid Customers Mix %] =
    VAR PaidCust =
        CALCULATE(
            DISTINCTCOUNT(fact_vCustomerAcquisition[CustomerKey]),
            dim_vAcquisitionChannel[IsOrganic] = 0
        )
    VAR TotalCust =
        DISTINCTCOUNT(fact_vCustomerAcquisition[CustomerKey])
    RETURN
        DIVIDE(PaidCust, TotalCust)
```

Pipeline P4 — Operational Efficiency

[Total Orders Fulfilled]

```
[Total Orders Fulfilled] =
    COUNTROWS(fact_vOrderFulfillment)
```

[On-Time Delivery Rate %]

```
[On-Time Delivery Rate %] =
    VAR OnTime =
        CALCULATE(
            COUNTROWS(fact_vOrderFulfillment),
            fact_vOrderFulfillment[IsOnTime] = 1
```

```
        )
    VAR Total = [Total Orders Fulfilled]
    RETURN
        DIVIDE(OnTime, Total)
```

[Avg Processing Days]

```
[Avg Processing Days] =
    AVERAGE(fact_vOrderFulfillment[ProcessingDays])
```

[Avg Transit Days]

```
[Avg Transit Days] =
    AVERAGE(fact_vOrderFulfillment[TransitDays])
```

[Avg Total Fulfillment Days]

```
[Avg Total Fulfillment Days] =
    AVERAGE(fact_vOrderFulfillment[TotalFulfillmentDays])
```

[SLA Attainment by Method]

```
[SLA Attainment by Method] =
    -- Applied in fact_vOrderFulfillment[ShippingMethod] filter context
    VAR OnTime =
        CALCULATE(
            COUNTROWS(fact_vOrderFulfillment),
            fact_vOrderFulfillment[IsOnTime] = 1
        )
    VAR Total = COUNTROWS(fact_vOrderFulfillment)
    RETURN
        DIVIDE(OnTime, Total)
```

Pipeline P5 — Returns & Refunds

[Total Return Events]

```
[Total Return Events] =
    COUNTROWS(fact_vReturns)
```

[Total Return Amount]

```
[Total Return Amount] =
    SUM(fact_vReturns[ReturnAmount])
```

[Return Rate % (Volume)]

```
[Return Rate % (Volume)] =
    VAR Returns = [Total Return Events]
    VAR Tx      = [Total Transactions]
    RETURN
        DIVIDE(Returns, Tx)
```

[Return Rate % (Value)]

```
[Return Rate % (Value)] =
    VAR RetAmt = [Total Return Amount]
    VAR Rev    = [Total Revenue]
    RETURN
        DIVIDE(RetAmt, Rev)
```

[Avg Return Lag Days]

```
[Avg Return Lag Days] =
    AVERAGE(fact_vReturns[ReturnLagDays])
```

[Operational Return Rate %]

```
[Operational Return Rate %] =
    VAR OpReturns =
        CALCULATE(
            [Total Return Events],
            dim_vReturnReason[IsOperationalFailure] = 1
        )
    VAR TotalRet = [Total Return Events]
    RETURN
        DIVIDE(OpReturns, TotalRet)
```

[Refund Amount]

```
[Refund Amount] =
    SUM(fact_vReturns[RefundAmount])
```

Pipeline P6 — Product & Category

[Total Units Sold]

```
[Total Units Sold] =
    VAR OnlineUnits = SUM(fact_vOnlineSales[SalesQuantity])
    VAR StoreUnits  = SUM(fact_vStoreSales[SalesQuantity])
    RETURN
        OnlineUnits + StoreUnits
```

[Product Return Rate %]

```
[Product Return Rate %] =
    VAR Returns = [Total Return Events]
    VAR Units   = [Total Units Sold]
    RETURN
        DIVIDE(Returns, Units)
```

[Inventory Turnover Rate]

```
[Inventory Turnover Rate] =
    VAR UnitsSold = [Total Units Sold]
    VAR AvgOnHand =
        CALCULATE(AVERAGE(fact_vInventory[OnHandQuantity]))
    RETURN
        DIVIDE(UnitsSold, AvgOnHand)
```

[Weeks of Supply]

```
[Weeks of Supply] =
    VAR AvgOnHand   =
        CALCULATE(AVERAGE(fact_vInventory[OnHandQuantity]))
    VAR WeeklySales = DIVIDE([Total Units Sold], 52)
    RETURN
        DIVIDE(AvgOnHand, WeeklySales)
```

[Stock-Out Events]

```
[Stock-Out Events] =
    CALCULATE(
        COUNTROWS(fact_vInventory),
        fact_vInventory[StockStatus] = "Out of Stock"
    )
```

[Sell-Through Rate %]

```
[Sell-Through Rate %] =
    VAR Sold   = [Total Units Sold]
    VAR OnHand =
        CALCULATE(AVERAGE(fact_vInventory[OnHandQuantity]))
    RETURN
        DIVIDE(Sold, Sold + OnHand)
```

[Revenue per Category]

```
[Revenue per Category] =
    -- Applied in dim_vProduct[CategoryName] filter context
    [Total Revenue]
```

Pipeline P7 — NPS & Satisfaction

[NPS Score]

```
[NPS Score] =
    VAR Promoters =
        CALCULATE(
            COUNTROWS(fact_vCustomerSurvey),
            fact_vCustomerSurvey[NPSCategory] = "Promoter"
        )
    VAR Detractors =
        CALCULATE(
            COUNTROWS(fact_vCustomerSurvey),
            fact_vCustomerSurvey[NPSCategory] = "Detractor"
        )
    VAR TotalResponses = COUNTROWS(fact_vCustomerSurvey)
    RETURN
        DIVIDE(Promoters - Detractors, TotalResponses) * 100
```

[Promoter %]

```
[Promoter %] =
    VAR Prom  =
        CALCULATE(
            COUNTROWS(fact_vCustomerSurvey),
```

```
            fact_vCustomerSurvey[NPSCategory] = "Promoter"
        )
    VAR Total = COUNTROWS(fact_vCustomerSurvey)
    RETURN
        DIVIDE(Prom, Total) * 100
```

[Detractor %]

```
[Detractor %] =
    VAR Det   =
        CALCULATE(
            COUNTROWS(fact_vCustomerSurvey),
            fact_vCustomerSurvey[NPSCategory] = "Detractor"
        )
    VAR Total = COUNTROWS(fact_vCustomerSurvey)
    RETURN
        DIVIDE(Det, Total) * 100
```

[Passive %]

```
[Passive %] =
    VAR Pass  =
        CALCULATE(
            COUNTROWS(fact_vCustomerSurvey),
            fact_vCustomerSurvey[NPSCategory] = "Passive"
        )
    VAR Total = COUNTROWS(fact_vCustomerSurvey)
    RETURN
        DIVIDE(Pass, Total) * 100
```

[Avg CSAT Score]

```
[Avg CSAT Score] =
    AVERAGE(fact_vCustomerSurvey[CSATScore])
```

[NPS by Acquisition Channel]

```
[NPS by Acquisition Channel] =
    -- Applied in dim_vAcquisitionChannel filter context
    VAR ChannelCusts =
        CALCULATETABLE(
            VALUES(fact_vCustomerAcquisition[CustomerKey])
        )
    RETURN
        CALCULATE(
            [NPS Score],
            TREATAS(
                ChannelCusts,
                fact_vCustomerSurvey[CustomerKey]
            )
        )
```